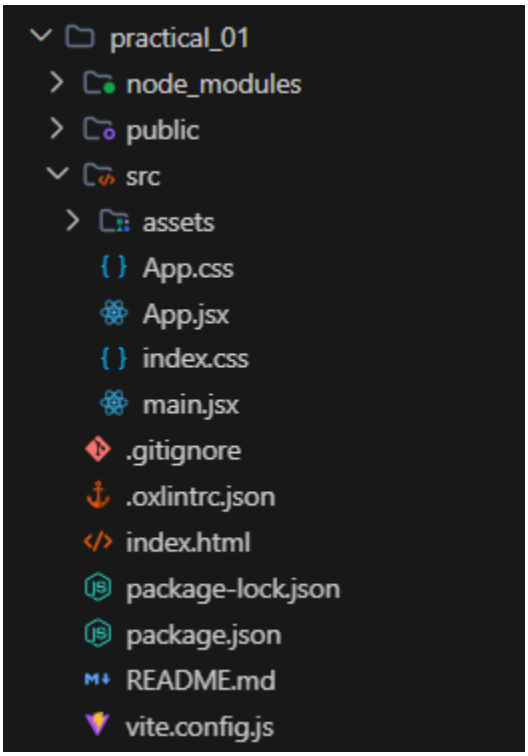
PRACTICAL NO. 01

Build a React Application using Functional Components, JSX and Component Reusability Concepts

Software Requirements

- Visual Studio Code
- Node.js and npm
- React
- Vite

Folder Structure



Source Code

1. App.jsx

```
import './App.css'

function MonumentCard({ name, state, builtBy, era }) {
  return (
    <div className="monument-card">
      <h2>{name}</h2>
      <p><strong>State:</strong> {state}</p>
      <p><strong>Built By:</strong> {builtBy}</p>
```

```
      <p><strong>Era:</strong> {era}</p>
    </div>
  )
}

function App() {
  return (
    <div className="app">
      <h1>Historical Monuments of India</h1>

      <div className="card-container">
        <MonumentCard
          name="Qutub Minar"
          state="Delhi"
          builtBy="Qutb-ud-din Aibak"
          era="Delhi Sultanate"
        />

        <MonumentCard
          name="Konark Sun Temple"
          state="Odisha"
          builtBy="King Narasimhadeva I"
          era="Eastern Ganga Dynasty"
        />

        <MonumentCard
          name="Mysore Palace"
          state="Karnataka"
          builtBy="Maharaja Krishnaraja Wadiyar IV"
          era="Wadiyar Dynasty"
        />
      </div>
    </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f3f4f6;
  color: #1f2937;
  min-height: 100vh;
}
```

```css
.app {
  min-height: 100vh;
  padding: 50px 20px;
  text-align: center;
  max-width: 1100px;
  margin: 0 auto;
}

.app h1 {
  margin-bottom: 35px;
  color: #0f766e;
  font-size: 2.2rem;
  font-weight: 700;
}

.card-container {
  display: flex;
  justify-content: center;
  gap: 25px;
  flex-wrap: wrap;
}

.monument-card {
  width: 290px;
  padding: 24px 20px;
  background: #ffffff;
  border-radius: 10px;
  border: 1px solid #e5e7eb;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
  text-align: left;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.monument-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.08);
}

.monument-card h2 {
  margin-top: 0;
  margin-bottom: 15px;
  font-size: 1.3rem;
  color: #111827;
  text-align: center;
  border-bottom: 1px solid #f3f4f6;
  padding-bottom: 10px;
}

.monument-card p {
  margin: 10px 0;
  font-size: 15px;
  color: #4b5563;
  line-height: 1.5;
```

```
}

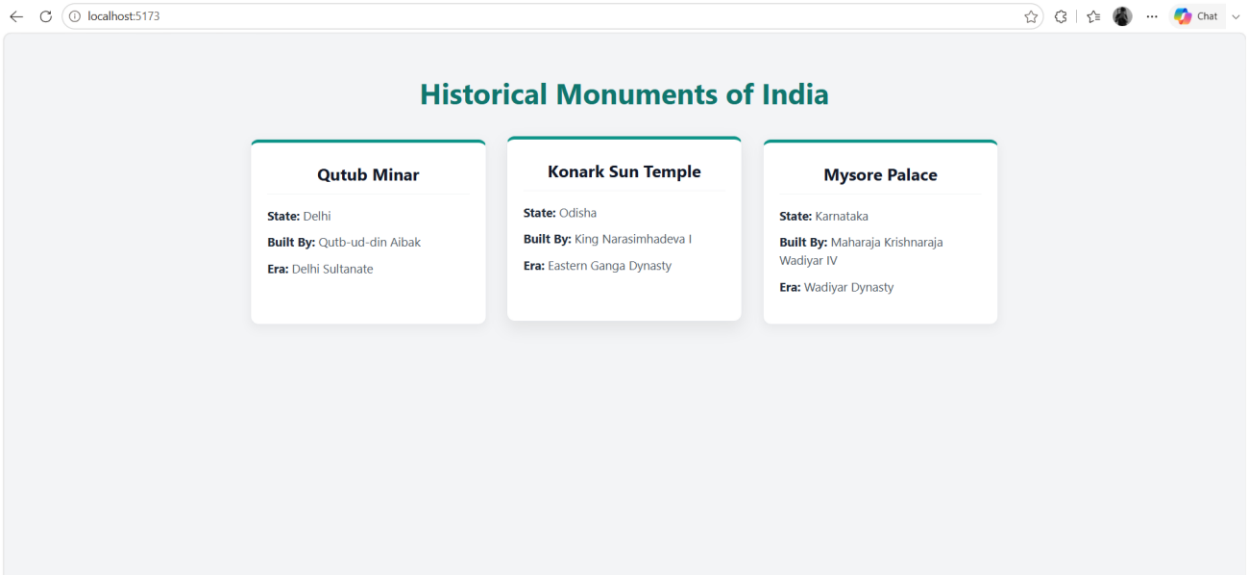
.monument-card p strong {
  color: #1f2937;
}
```

3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, the React application was successfully developed using Functional Components, JSX and Component Reusability concepts.

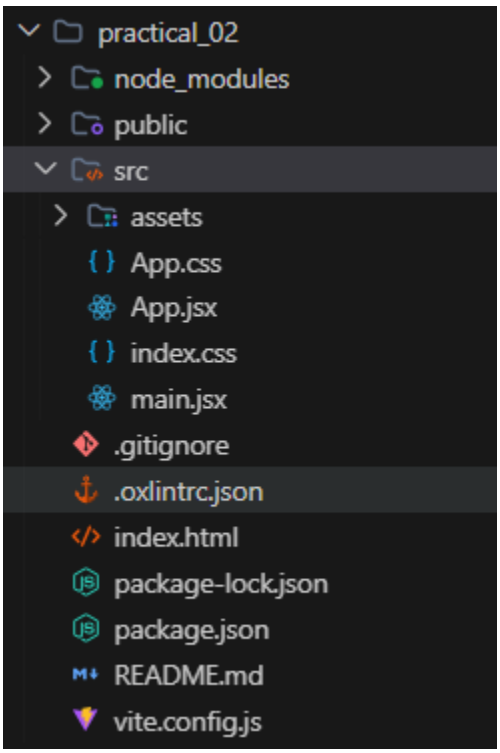
PRACTICAL NO. 02

Develop a React application implementing Props, State Management and Event Handling.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite

Folder Structure



Source Code

1. App.jsx

```
import { useState } from 'react'
import './App.css'

function HandicraftCard({ itemName, origin, price }) {
  const [count, setCount] = useState(0)

  const handleIncrement = () => {
    setCount(count + 1)
  }
```

```
  const handleDecrement = () => {
    if (count > 0) {
      setCount(count - 1)
    }
  }

  const handleOrder = () => {
    if (count > 0) {
      alert(`Order placed for ${count} piece(s) of ${itemName}!`)
    } else {
      alert('Please select quantity first.')
    }
  }

  return (
    <div className="product-card">
      <h2>{itemName}</h2>
      <p className="origin"><strong>Origin:</strong> {origin}</p>
      <p className="price">₹{price}</p>

      <div className="quantity-section">
        <button onClick={handleDecrement}>−</button>
        <span>{count}</span>
        <button onClick={handleIncrement}>+</button>
      </div>

      <p className="total-text">
        Total: <strong>₹{price * count}</strong>
      </p>

      <button className="cart-button" onClick={handleOrder}>
        Add to Cart
      </button>
    </div>
  )
}

function App() {
  return (
    <div className="app">
      <h1>Traditional Handicrafts Store</h1>
      <p className="subtitle">
        Props, State Management and Event Handling Demo
      </p>

      <div className="product-container">
        <HandicraftCard
          itemName="Banarasi Silk Saree"
          origin="Varanasi (UP)"
          price={3200}
        />

        <HandicraftCard
          itemName="Dhokra Brass Figurine"
```

```
          origin="Bastar (Chhattisgarh)"
          price={1250}
        />

        <HandicraftCard
          itemName="Channapatna Wooden Toys"
          origin="Karnataka"
          price={650}
        />
      </div>
    </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #f8fafc;
  color: #334155;
  min-height: 100vh;
}

.app {
  min-height: 100vh;
  padding: 50px 20px;
  text-align: center;
  max-width: 1100px;
  margin: 0 auto;
}

.app h1 {
  margin-bottom: 10px;
  color: #1e3a8a;
  font-size: 2.2rem;
  font-weight: 700;
}

.subtitle {
  color: #64748b;
  margin-bottom: 35px;
  font-size: 1rem;
}
```

```
.product-container {
  display: flex;
  justify-content: center;
  gap: 25px;
  flex-wrap: wrap;
}

.product-card {
  width: 300px;
  padding: 25px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.06);
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0284c7;
  text-align: left;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.product-card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.1);
}

.product-card h2 {
  margin-top: 0;
  margin-bottom: 10px;
  font-size: 1.25rem;
  color: #0f172a;
  text-align: center;
}

.origin {
  font-size: 0.9rem;
  color: #64748b;
  margin-bottom: 12px;
  text-align: center;
}

.price {
  font-size: 1.25rem;
  color: #059669;
  font-weight: bold;
  margin-bottom: 15px;
  text-align: center;
}

.quantity-section {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 15px;
  margin-bottom: 15px;
  background: #f1f5f9;
  padding: 8px;
```

```
  border-radius: 8px;
}

.quantity-section button {
  background-color: #0284c7;
  color: white;
  border: none;
  width: 32px;
  height: 32px;
  border-radius: 50%;
  font-size: 16px;
  cursor: pointer;
  transition: background-color 0.2s;
  display: flex;
  align-items: center;
  justify-content: center;
}

.quantity-section button:hover {
  background-color: #0369a1;
}

.quantity-section span {
  font-size: 1.1rem;
  font-weight: bold;
  min-width: 24px;
  text-align: center;
  color: #0f172a;
}

.total-text {
  text-align: center;
  margin-bottom: 15px;
  font-size: 1rem;
  color: #334155;
}

.cart-button {
  width: 100%;
  padding: 11px;
  background-color: #059669;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}

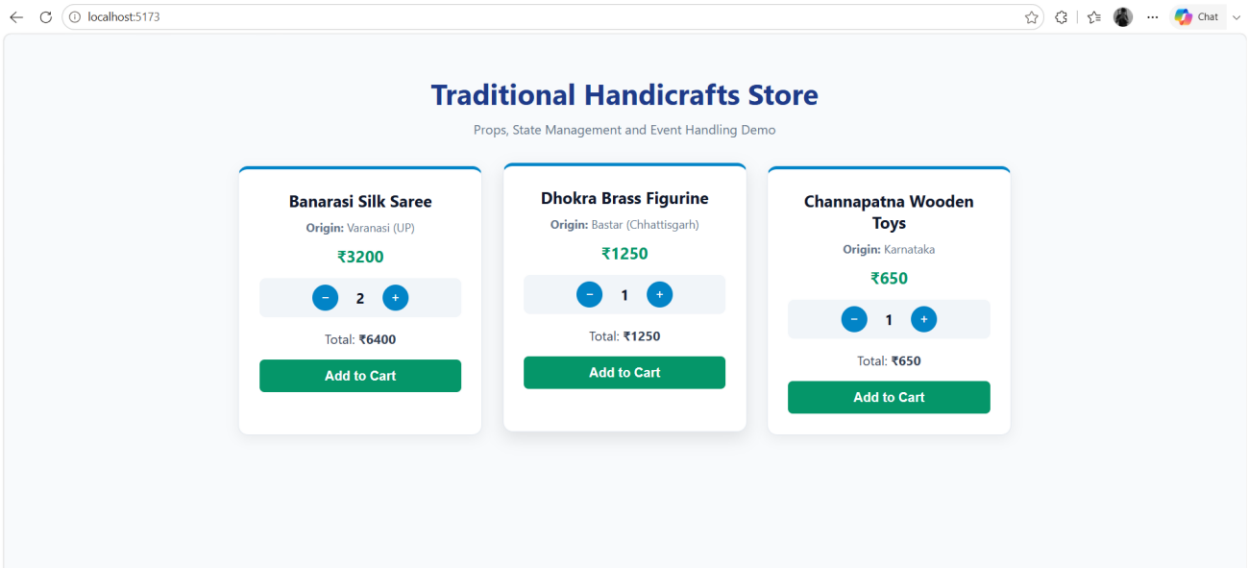
.cart-button:hover {
  background-color: #047857;
}
```

3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, the React application was successfully developed using Props, State Management, and Event Handling concepts. The application successfully handled user interactions and dynamically updated the user interface using React state.

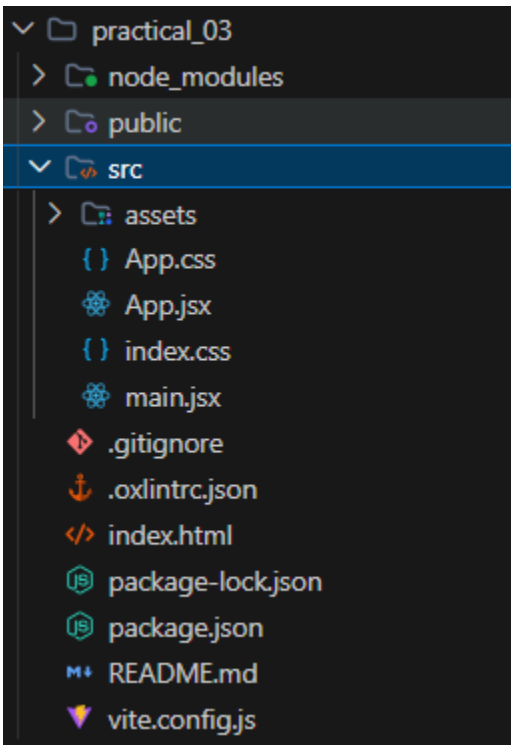
PRACTICAL NO. 03

Create a Student Management System using React Forms, Controlled Components and Validation.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite

Folder Structure



Source Code

1. App.jsx

```
import { useState } from 'react'
import './App.css'

function App() {
  const [student, setStudent] = useState({
    name: '',
    email: '',
    course: '',
    age: '',
```

```
})

const [students, setStudents] = useState([])
const [errors, setErrors] = useState({})

const handleChange = (event) => {
  const { name, value } = event.target
  setStudent({
    ...student,
    [name]: value,
  })
}

const validateForm = () => {
  const newErrors = {}

  if (student.name.trim() === '') {
    newErrors.name = 'Student name is required.'
  }

  if (student.email.trim() === '') {
    newErrors.email = 'Email is required.'
  } else if (!student.email.includes('@')) {
    newErrors.email = 'Enter a valid email address.'
  }

  if (student.course === '') {
    newErrors.course = 'Please select a course.'
  }

  if (student.age === '') {
    newErrors.age = 'Age is required.'
  } else if (student.age < 17 || student.age > 40) {
    newErrors.age = 'Age must be between 17 and 40.'
  }

  setErrors(newErrors)
  return Object.keys(newErrors).length === 0
}

const handleSubmit = (event) => {
  event.preventDefault()

  if (validateForm()) {
    const newStudent = {
      id: Date.now(),
      ...student,
    }

    setStudents([...students, newStudent])
    setStudent({
      name: '',
      email: '',
      course: '',
      age: '',
```

```
    })
    setErrors({})
    alert('Student registered successfully!')
  }
}

const deleteStudent = (id) => {
  setStudents(students.filter((s) => s.id !== id))
}

return (
  <div className="app">
    <h1>Student Information System</h1>

    <div className="form-container">
      <h2>Register New Student</h2>

      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label>Student Name</label>
          <input
            type="text"
            name="name"
            value={student.name}
            onChange={handleChange}
            placeholder="Enter full name"
          />
          {errors.name && <p className="error">{errors.name}</p>}
        </div>

        <div className="form-group">
          <label>Email Address</label>
          <input
            type="email"
            name="email"
            value={student.email}
            onChange={handleChange}
            placeholder="Enter email address"
          />
          {errors.email && <p className="error">{errors.email}</p>}
        </div>

        <div className="form-group">
          <label>Course</label>
          <select
            name="course"
            value={student.course}
            onChange={handleChange}
          >
            <option value="">Select Course</option>
            <option value="M.Sc. CA & IT">M.Sc. CA & IT</option>
            <option value="B.Sc. CA & IT">B.Sc. CA & IT</option>
            <option value="BCA">BCA</option>
            <option value="MCA">MCA</option>
          </select>
```

```
      {errors.course && <p className="error">{errors.course}</p>}
    </div>

    <div className="form-group">
      <label>Age</label>
      <input
        type="number"
        name="age"
        value={student.age}
        onChange={handleChange}
        placeholder="Enter student age"
      />
      {errors.age && <p className="error">{errors.age}</p>}
    </div>

    <button type="submit" className="add-button">
      Save Student
    </button>
  </form>
</div>

<div className="student-list">
  <h2>Registered Students Record</h2>

  {students.length === 0 ? (
    <p className="empty-message">No student records found.</p>
  ) : (
    <div className="table-container">
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
            <th>Age</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {students.map((s) => (
            <tr key={s.id}>
              <td>{s.name}</td>
              <td>{s.email}</td>
              <td>{s.course}</td>
              <td>{s.age}</td>
              <td>
                <button
                  className="delete-button"
                  onClick={() => deleteStudent(s.id)}
                >
                  Delete
                </button>
              </td>
            </tr>
          ))}
```

```
            </tbody>
          </table>
        </div>
      )}
    </div>
  </div>
 )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  max-width: 900px;
  margin: 0 auto;
  padding: 40px 20px;
}

.app h1 {
  text-align: center;
  color: #0f766e;
  margin-bottom: 30px;
  font-size: 2.2rem;
}

.form-container, .student-list {
  background: #ffffff;
  padding: 30px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05);
  margin-bottom: 30px;
}

.form-container h2, .student-list h2 {
  font-size: 1.35rem;
  color: #1e293b;
  margin-bottom: 20px;
```

```
  border-bottom: 2px solid #f1f5f9;
  padding-bottom: 10px;
}

.form-group {
  margin-bottom: 18px;
}

.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 8px;
  color: #475569;
  font-size: 0.95rem;
}

.form-group input,
.form-group select {
  width: 100%;
  padding: 10px 12px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 1rem;
  outline: none;
  background: #fff;
  transition: border-color 0.2s;
}

.form-group input:focus,
.form-group select:focus {
  border-color: #0d9488;
}

.error {
  color: #ef4444;
  font-size: 0.85rem;
  margin-top: 5px;
}

.add-button {
  width: 100%;
  padding: 12px;
  background-color: #0d9488;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.2s;
}

.add-button:hover {
  background-color: #0f766e;
}
```

```css
.empty-message {
  color: #94a3b8;
  font-style: italic;
  text-align: center;
  padding: 20px 0;
}

.table-container {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 10px;
  text-align: left;
}

th, td {
  padding: 12px 14px;
  border-bottom: 1px solid #e2e8f0;
  font-size: 0.95rem;
}

th {
  background-color: #f1f5f9;
  color: #1e293b;
  font-weight: 600;
}

tr:hover {
  background-color: #f8fafc;
}

.delete-button {
  background-color: #ef4444;
  color: white;
  border: none;
  padding: 6px 14px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
  transition: background-color 0.2s;
}

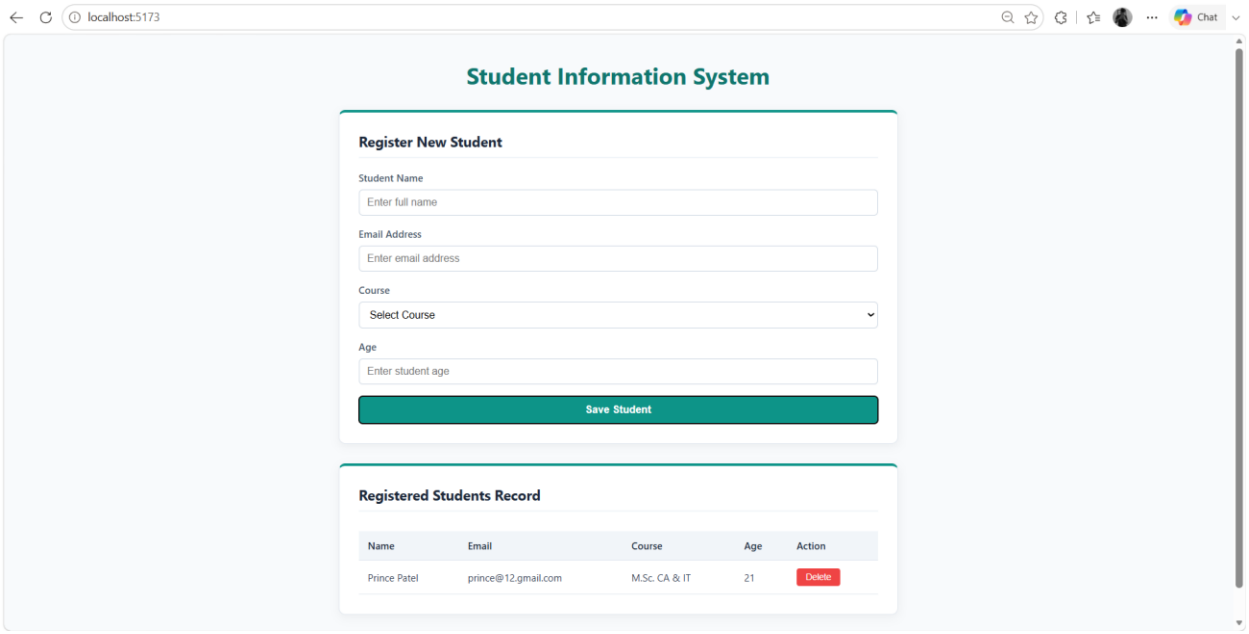
.delete-button:hover {
  background-color: #dc2626;
}
```

3. main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output



Result

Thus, a Student Management System was successfully developed using React Forms, Controlled Components and Validation. The application successfully accepts student information, validates the user input and manages form data using React state.

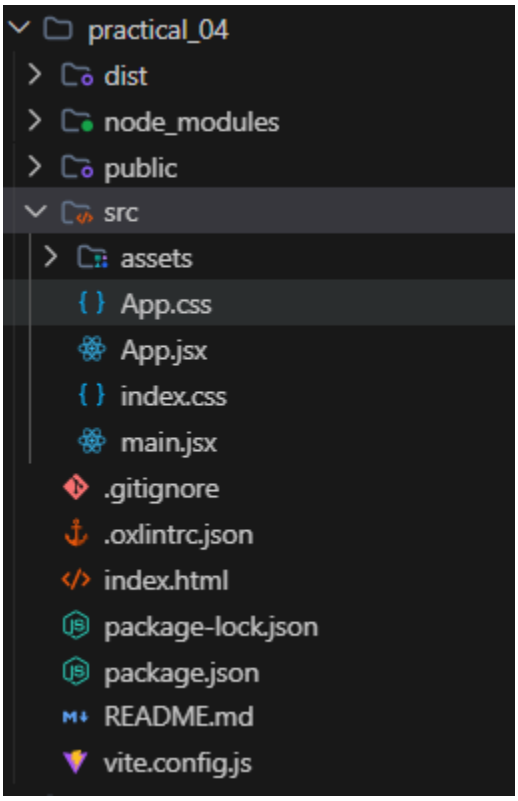
PRACTICAL NO. 04

Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Routing Library: React Router DOM
- Build Tool: Vite
- Web Browser: Google Chrome

Folder Structure



Source Code

1. App.jsx

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom'
import './App.css'

function Home() {
```

```
  return (
    <div className="page">
      <h1>Welcome to Bharat Tourism Portal</h1>
      <p>
        Explore the vibrant heritage, iconic monuments, and diverse cultural landscapes
across India.
      </p>
      <p>
        Navigate seamlessly using the menu above to discover famous monuments, prominent
regions, and platform details.
      </p>
    </div>
  )
}

function Monuments() {
  const monumentsList = [
    { name: 'Red Fort', state: 'Delhi' },
    { name: 'Meenakshi Temple', state: 'Tamil Nadu' },
    { name: 'Victoria Memorial', state: 'West Bengal' },
  ]

  return (
    <div className="page">
      <h1>Heritage Monuments</h1>

      <div className="card-container">
        {monumentsList.map((item, index) => (
          <div className="card" key={index}>
            <h3>{item.name}</h3>
            <p>State: {item.state}</p>
          </div>
        ))}
      </div>
    </div>
  )
}

function Regions() {
  return (
    <div className="page">
      <h1>Popular Destinations</h1>

      <div className="card-container">
        <div className="card">
          <h3>Delhi</h3>
          <p>Capital territory showcasing centuries of rich dynastic history and modern
monuments.</p>
        </div>

        <div className="card">
          <h3>Tamil Nadu</h3>
          <p>Cradle of Dravidian architecture, grand temple towers, and classical
arts.</p>
        </div>
```

```
        <div className="card">
          <h3>West Bengal</h3>
          <p>Cultural hub known for colonial architecture, literature, and living
traditions.</p>
        </div>
      </div>
    </div>
  )
}

function About() {
  return (
    <div className="page">
      <h1>About This Portal</h1>

      <p>
        This Single Page Application (SPA) is built using React and React Router for
seamless navigation without full page reload.
      </p>

      <p>
        It showcases declarative routing, NavLink active state management, and clean
modular component architecture.
      </p>
    </div>
  )
}

function NotFound() {
  return (
    <div className="page">
      <h1>404</h1>
      <p>Page Not Found</p>
    </div>
  )
}

function App() {
  return (
    <BrowserRouter>
      <div className="app">
        <nav className="navbar">
          <h2>Bharat Tourism Portal</h2>

          <div className="nav-links">
            <NavLink to="/">Home</NavLink>
            <NavLink to="/monuments">Monuments</NavLink>
            <NavLink to="/regions">Regions</NavLink>
            <NavLink to="/about">About</NavLink>
          </div>
        </nav>

        <main>
          <Routes>
```

```
            <Route path="/" element={<Home />} />
            <Route path="/monuments" element={<Monuments />} />
            <Route path="/regions" element={<Regions />} />
            <Route path="/about" element={<About />} />
            <Route path="*" element={<NotFound />} />
          </Routes>
        </main>
      </div>
    </BrowserRouter>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.navbar {
  background-color: #ffffff;
  padding: 18px 40px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.05);
  border-bottom: 1px solid #e2e8f0;
}

.navbar h2 {
  color: #0f766e;
  font-size: 1.45rem;
}

.nav-links {
  display: flex;
  gap: 15px;
```

```
}

.nav-links a {
  text-decoration: none;
  color: #475569;
  font-weight: 600;
  font-size: 0.95rem;
  padding: 8px 16px;
  border-radius: 6px;
  transition: all 0.2s ease;
}

.nav-links a:hover {
  color: #0d9488;
  background-color: #f0fdfa;
}

.nav-links a.active {
  color: #ffffff;
  background-color: #0d9488;
}

main {
  flex: 1;
  padding: 40px 20px;
  max-width: 1050px;
  margin: 0 auto;
  width: 100%;
}

.page {
  background: #ffffff;
  padding: 40px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.04);
  text-align: center;
}

.page h1 {
  color: #0f766e;
  margin-bottom: 20px;
  font-size: 2.1rem;
}

.page p {
  color: #475569;
  font-size: 1.05rem;
  line-height: 1.6;
  margin-bottom: 15px;
}

.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
```

```
  gap: 20px;
  margin-top: 30px;
  text-align: left;
}

.card {
  background: #f8fafc;
  padding: 22px;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
  border-left: 4px solid #0d9488;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.03);
}

.card h3 {
  color: #1e293b;
  margin-bottom: 8px;
  font-size: 1.25rem;
}

.card p {
  font-size: 0.95rem;
  color: #64748b;
  margin: 0;
}
```

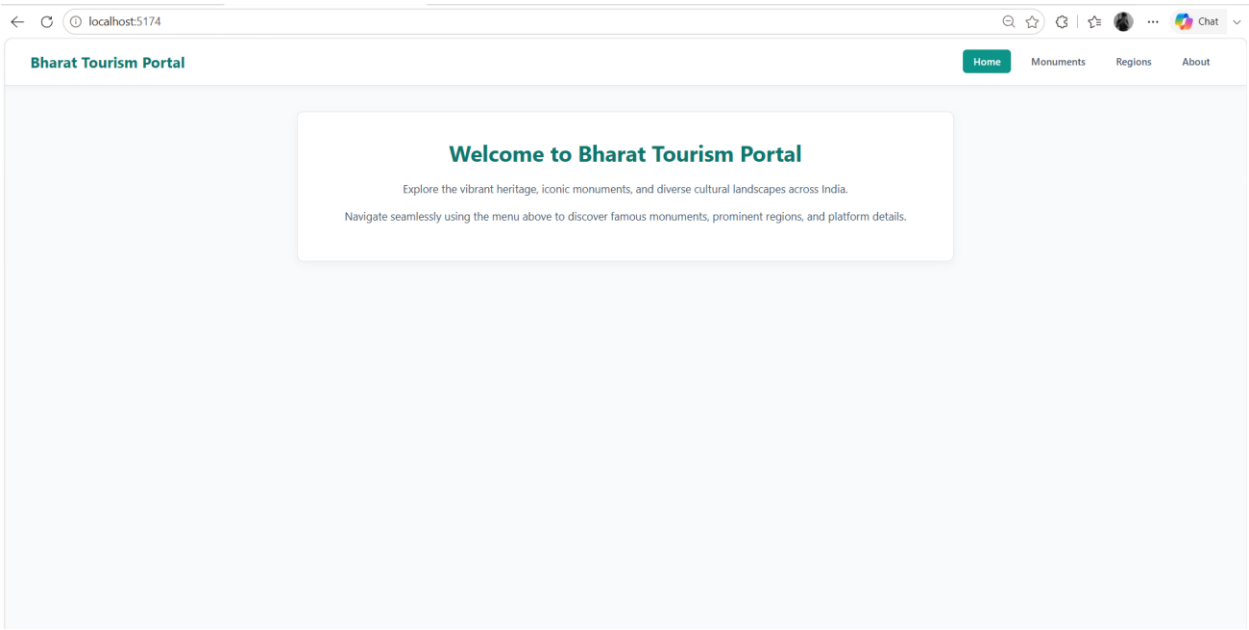
3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

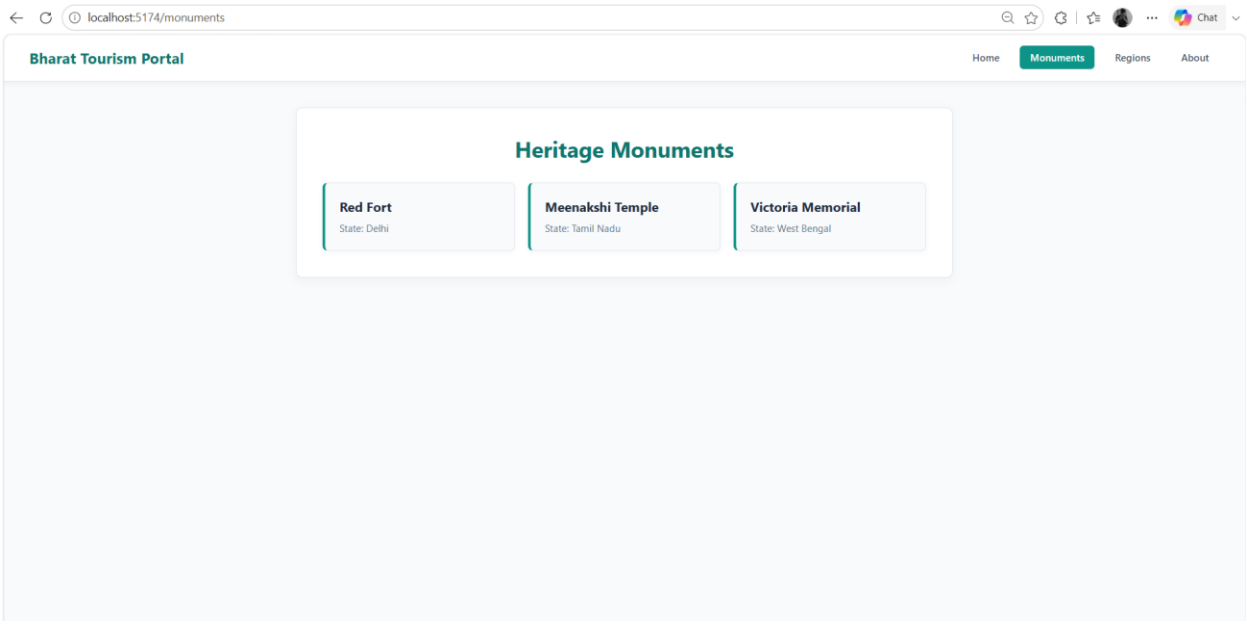
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output

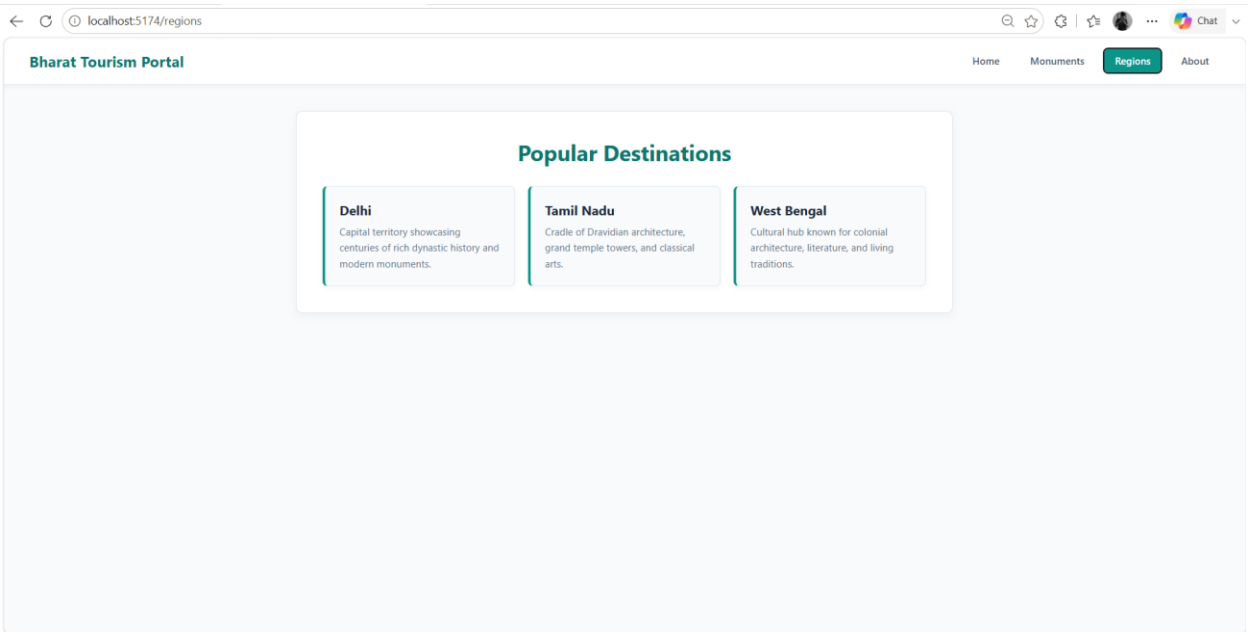
1) Home Page



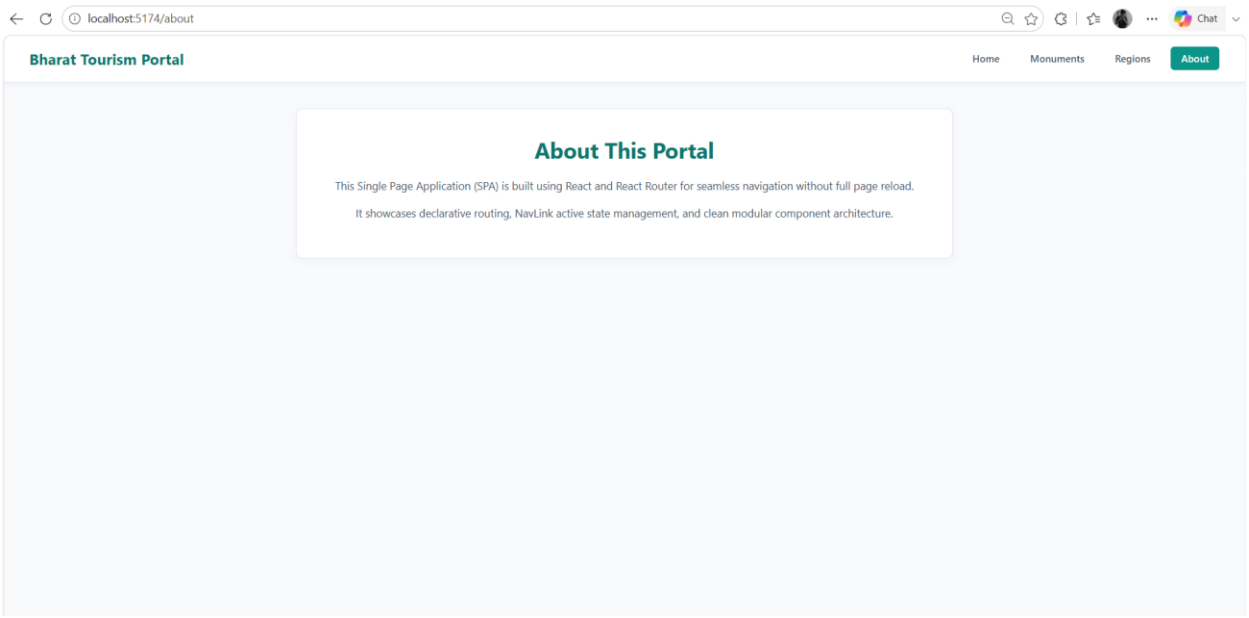
2) Monuments Page



3) Regions Page



4) About Page



Result

Thus, a Single Page Application (SPA) was successfully developed using React Router. The application provides navigation between multiple pages without reloading the complete web page and successfully demonstrates BrowserRouter, Routes, Route and NavLink concepts.

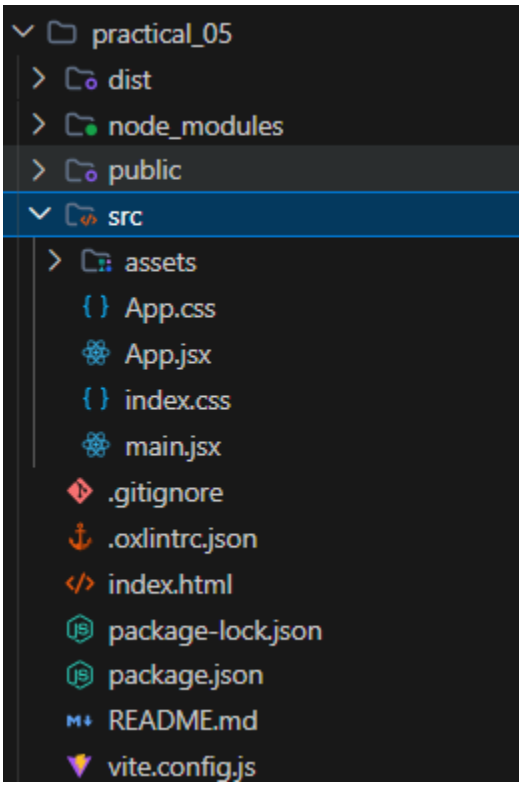
PRACTICAL NO. 05

Develop a React application consuming REST APIs using Axios and Fetch API.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- JavaScript Library: React.js
- Build Tool: Vite
- HTTP Client Library: Axios
- REST API: JSONPlaceholder & DummyJSON
- Web Browser: Google Chrome

Folder Structure



Source Code

1. App.jsx

```
import { useState, useEffect } from 'react'
import axios from 'axios'
import './App.css'
```

```
function App() {
  const [teamMembers, setTeamMembers] = useState([]) // Using Fetch API
  const [products, setProducts] = useState([]) // Using Axios
  const [loadingMembers, setLoadingMembers] = useState(true)
  const [loadingProducts, setLoadingProducts] = useState(true)

  // 1. Fetch API - Team Members Data
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/users')
      .then((response) => response.json())
      .then((data) => {
        const teamNames = ['Aditya Kulkarni', 'Ananya Deshmukh', 'Vikramaditya Rao',
'Ishita Sengupta']
        const engineeringTeam = data.slice(0, 4).map((user, index) => ({
          id: user.id,
          name: teamNames[index],
          role: ['Cloud Architect', 'Frontend Engineer', 'API Specialist', 'Data
Engineer'][index],
          email: user.email,
        }))
        setTeamMembers(engineeringTeam)
        setLoadingMembers(false)
      })
      .catch((error) => console.error('Error with Fetch API:', error))
  }, [])

  // 2. Axios - Featured Products Data
  useEffect(() => {
    axios
      .get('https://dummyjson.com/products?limit=4')
      .then((response) => {
        setProducts(response.data.products)
        setLoadingProducts(false)
      })
      .catch((error) => console.error('Error with Axios:', error))
  }, [])

  return (
    <div className="app">
      <h1>REST API Integration Dashboard</h1>
      <p className="subtitle">Consuming REST APIs using Fetch API and Axios</p>

      <section className="api-section">
        <h2>Engineering Team (Retrieved via Fetch API)</h2>
        {loadingMembers ? (
          <p className="loading">Loading team members...</p>
        ) : (
          <div className="card-container">
            {teamMembers.map((member) => (
              <div className="card" key={member.id}>
                <h3>{member.name}</h3>
                <p><strong>Role:</strong> {member.role}</p>
                <p><strong>Email:</strong> {member.email}</p>
              </div>
            ))}
```

```
          </div>
        )}
      </section>

      <section className="api-section">
        <h2>Featured Products (Retrieved via Axios)</h2>
        {loadingProducts ? (
          <p className="loading">Loading products...</p>
        ) : (
          <div className="card-container">
            {products.map((item) => (
              <div className="card" key={item.id}>
                <h3>{item.title}</h3>
                <p className="price">₹{(item.price * 80).toFixed(0)}</p>
                <p><strong>Category:</strong> {item.category}</p>
              </div>
            ))}
          </div>
        )}
      </section>
    </div>
  )
}

export default App
```

2. App.css

```
#root {
  width: 100%;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f8fafc;
  color: #334155;
}

.app {
  max-width: 1100px;
  margin: 0 auto;
  padding: 40px 20px;
}

.app h1 {
  text-align: center;
  color: #0f766e;
  font-size: 2.2rem;
  margin-bottom: 5px;
}
```

```
.subtitle {
  text-align: center;
  color: #64748b;
  margin-bottom: 35px;
  font-size: 1rem;
}

.api-section {
  background: #ffffff;
  padding: 30px;
  border-radius: 10px;
  border: 1px solid #e2e8f0;
  border-top: 4px solid #0d9488;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.04);
  margin-bottom: 30px;
}

.api-section h2 {
  font-size: 1.3rem;
  color: #1e293b;
  margin-bottom: 20px;
  border-bottom: 2px solid #f1f5f9;
  padding-bottom: 10px;
}

.card-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

.card {
  background: #f8fafc;
  padding: 20px;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
  border-left: 4px solid #0d9488;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.03);
}

.card h3 {
  font-size: 1.1rem;
  color: #1e293b;
  margin-bottom: 10px;
}

.card p {
  font-size: 0.9rem;
  color: #475569;
  margin-bottom: 6px;
}

.price {
  color: #059669 !important;
```

```
  font-weight: bold;
  font-size: 1.05rem;
}

.loading {
  text-align: center;
  color: #94a3b8;
  font-style: italic;
  padding: 20px 0;
}
```

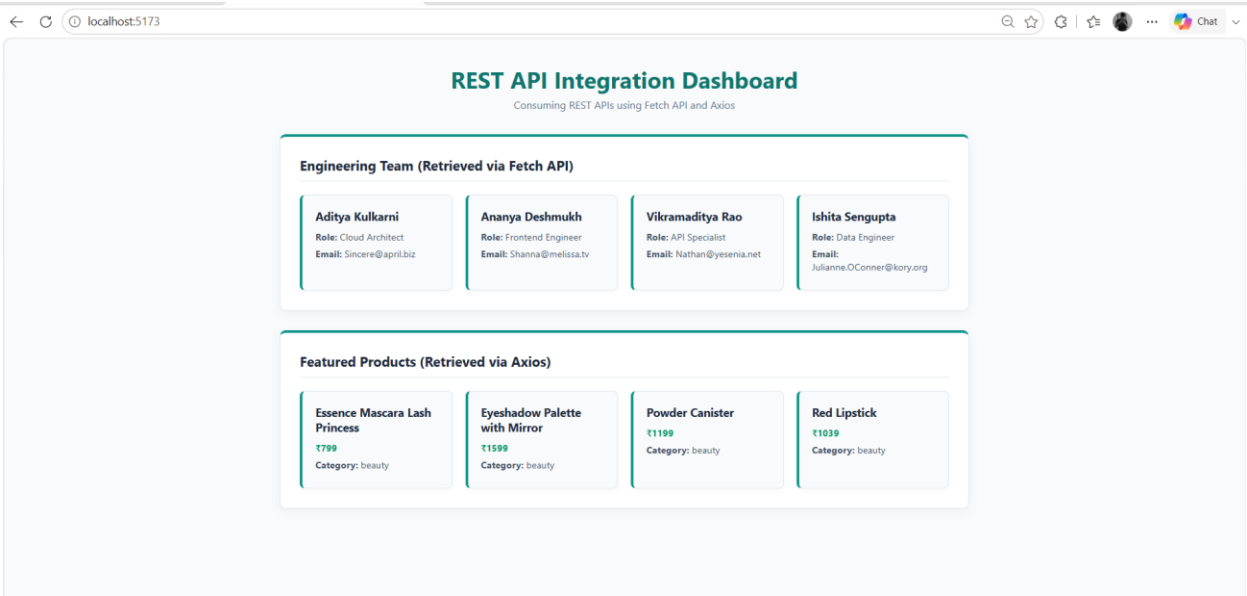
3. main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Output

Final Output Screenshot:
1) Fetch API & Axios section



Result

Thus, a React application was successfully developed to consume REST APIs using Fetch API and Axios. The application successfully retrieved and displayed user data from a REST API while demonstrating state management, side effects, loading states and error handling.

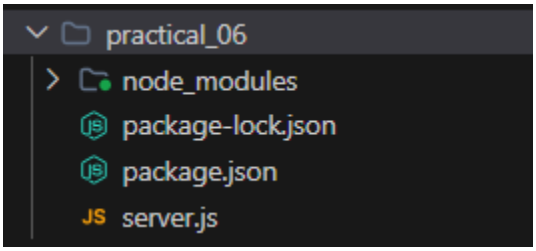
PRACTICAL NO. 06

Create a backend server using Express.js implementing routing, middleware and request handling.

Software Requirements

- Code Editor: Visual Studio Code
- Runtime Environment: Node.js
- Package Manager: npm
- Backend Framework: Express.js
- API Testing Tool: Google Chrome / Postman
- Web Browser: Google Chrome

Folder Structure



Source Code

1. server.js

```
const express = require('express')

const app = express()

const PORT = 5000


app.use(express.json())


const requestLogger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`)
  next()
}

app.use(requestLogger)


app.get('/', (req, res) => {
  res.json({
    message: 'Welcome to Student Management API'
  })
})
```

```
app.get('/api/students', (req, res) => {
  const students = [
    {
      id: 1,
      name: 'Aryan Joshi',
      course: 'M.Sc. CA & IT'
    },
    {
      id: 2,
      name: 'Divya Mehta',
      course: 'M.Sc. CA & IT'
    },
    {
      id: 3,
      name: 'Karan Sharma',
      course: 'MCA'
    }
  ]

  res.json(students)
})


app.get('/api/students/:id', (req, res) => {
  const studentId = req.params.id

  res.json({
    message: 'Student requested successfully',
    studentId: studentId
  })
})


app.post('/api/students', (req, res) => {
  const student = req.body

  res.status(201).json({
    message: 'Student added successfully',
    student: student
  })
})


app.use((req, res) => {
  res.status(404).json({
    message: 'Route not found'
  })
})


app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`)
})
```

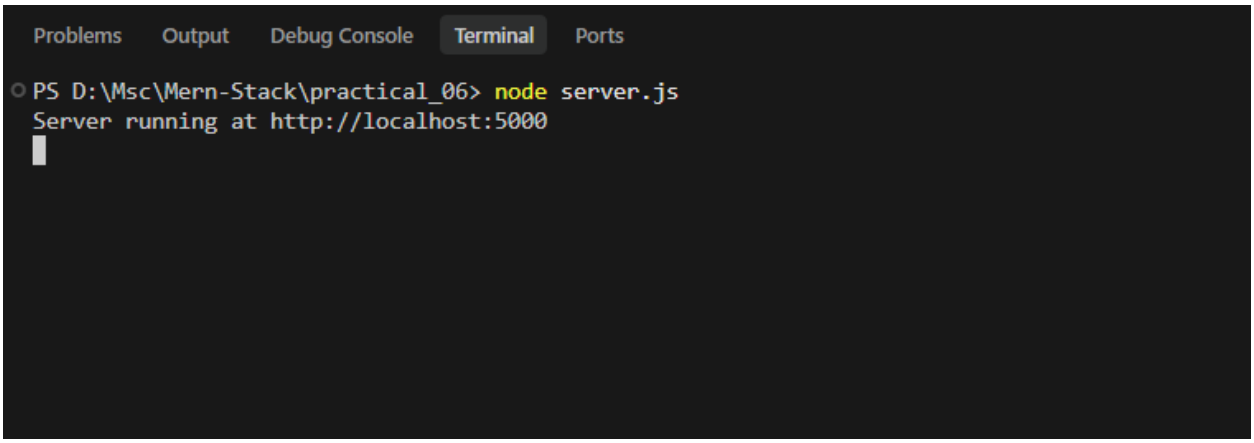
2. package.json

```
{
  "name": "practical_06",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

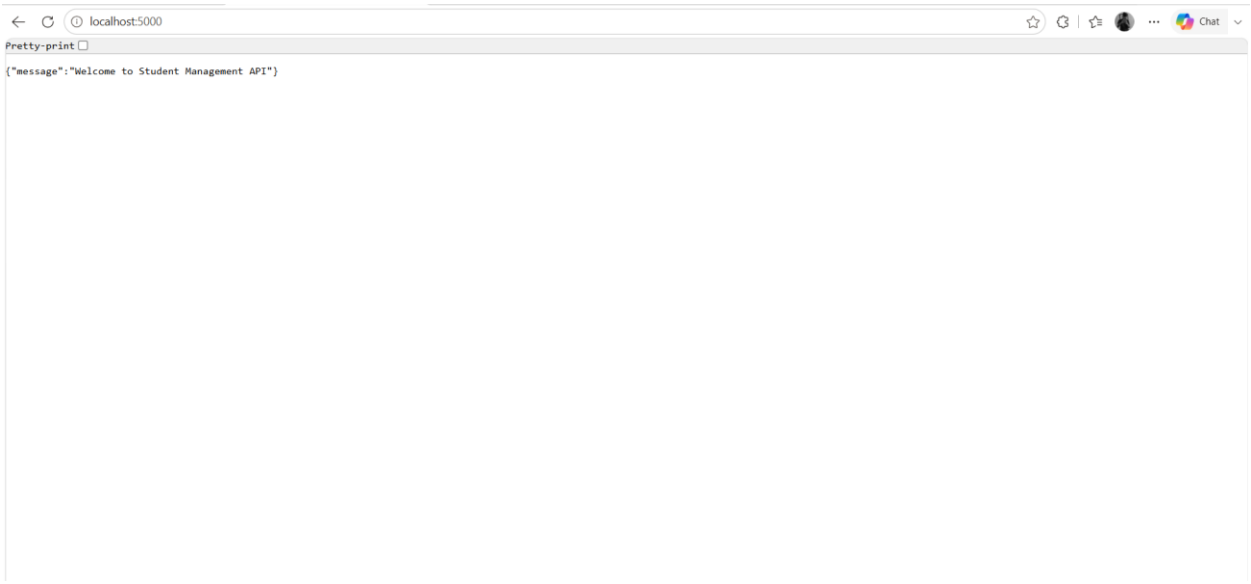
Output

Final Output Screenshot:

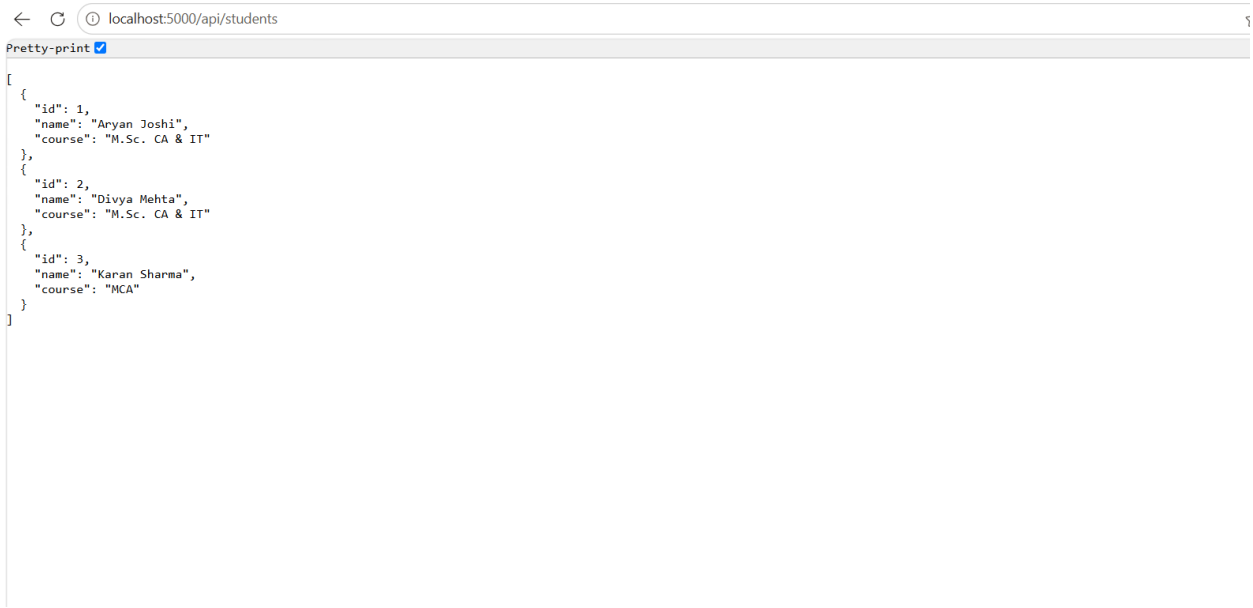
1) Terminal



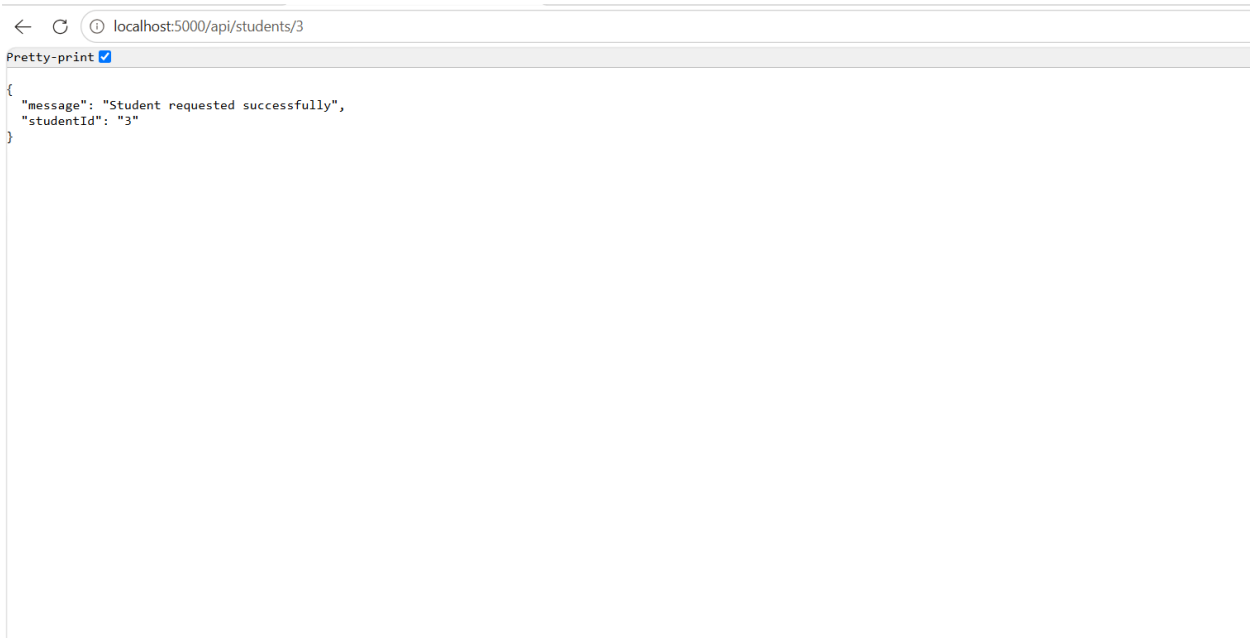
2) Browser Home Route



3) Browser Students Route



4) Browser Single Student Route



Result

Thus, a backend server was successfully created using Express.js. The application successfully demonstrated routing, middleware and request handling by processing GET and POST requests and sending JSON responses to the client.

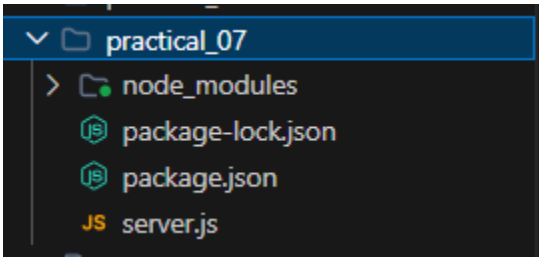
PRACTICAL NO. 07

Build RESTful APIs for Student Records Management using Express.js and Postman testing.

Software Requirements

- Visual Studio Code
- Node.js
- npm (Node Package Manager)
- Express.js
- Postman
- Google Chrome

Folder Structure



Source Code

1. server.js

```
const express = require('express')

const app = express()

const PORT = 5000


app.use(express.json())


let students = [
  {
    id: 1,
    name: 'Aryan Joshi',
    email: 'aryan@gmail.com',
    course: 'M.Sc. CA & IT'
  },
  {
    id: 2,
    name: 'Divya Mehta',
    email: 'divya@gmail.com',
    course: 'M.Sc. CA & IT'
  },
```

```
  {
    id: 3,
    name: 'Karan Sharma',
    email: 'karan@gmail.com',
    course: 'MCA'
  }
]


app.get('/', (req, res) => {
  res.json({
    message: 'Student Records REST API is running'
  })
})


app.get('/api/students', (req, res) => {
  res.json(students)
})


app.get('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const student = students.find(
    (student) => student.id === id
  )

  if (!student) {
    return res.status(404).json({
      message: 'Student not found'
    })
  }

  res.json(student)
})


app.post('/api/students', (req, res) => {
  const { name, email, course } = req.body

  if (!name || !email || !course) {
    return res.status(400).json({
      message: 'Name, email and course are required'
    })
  }

  const newStudent = {
    id: students.length + 1,
    name,
    email,
    course
  }

  students.push(newStudent)
```

```
  res.status(201).json({
    message: 'Student added successfully',
    student: newStudent
  })
})


app.put('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const student = students.find(
    (student) => student.id === id
  )

  if (!student) {
    return res.status(404).json({
      message: 'Student not found'
    })
  }

  const { name, email, course } = req.body

  if (name) student.name = name
  if (email) student.email = email
  if (course) student.course = course

  res.json({
    message: 'Student updated successfully',
    student
  })
})


app.delete('/api/students/:id', (req, res) => {
  const id = parseInt(req.params.id)

  const studentIndex = students.findIndex(
    (student) => student.id === id
  )

  if (studentIndex === -1) {
    return res.status(404).json({
      message: 'Student not found'
    })
  }

  const deletedStudent = students.splice(
    studentIndex,
    1
  )

  res.json({
    message: 'Student deleted successfully',
    student: deletedStudent[0]
```

```
  })
})


app.use((req, res) => {
  res.status(404).json({
    message: 'Route not found'
  })
})


app.listen(PORT, () => {
  console.log(
    `Server running at http://localhost:${PORT}`
  )
})
```

2. package.json

```
{
  "name": "practical_07",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

Output

Final Output Screenshot:
1) Terminal Screenshot

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Msc\Mern-Stack\practical_07> node server.js
Server running at http://localhost:5000
```

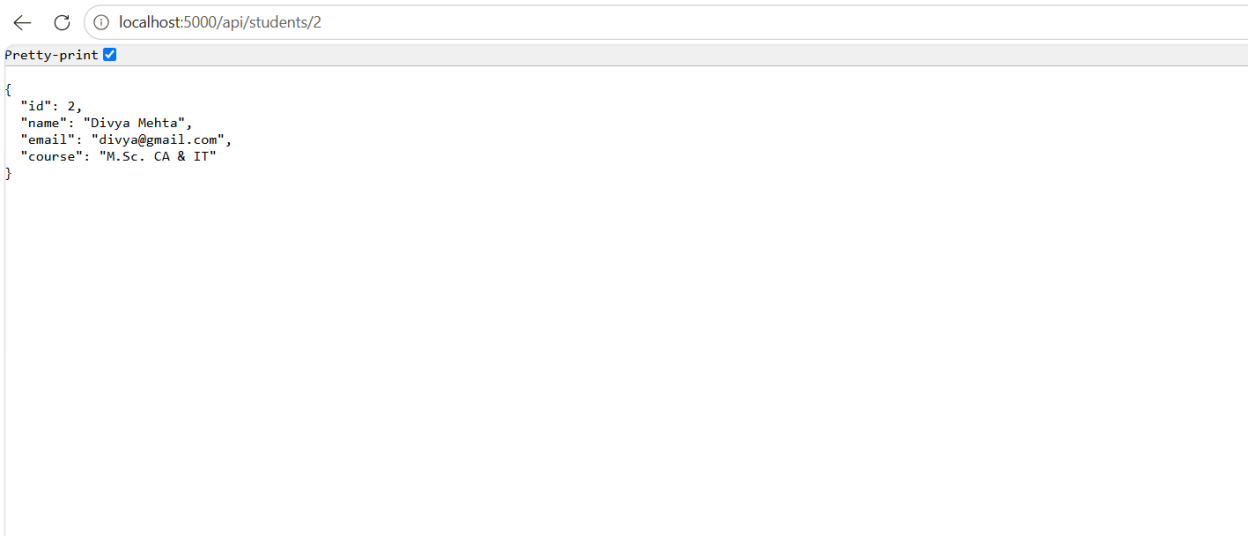
2) GET All Students

localhost:5000/api/students

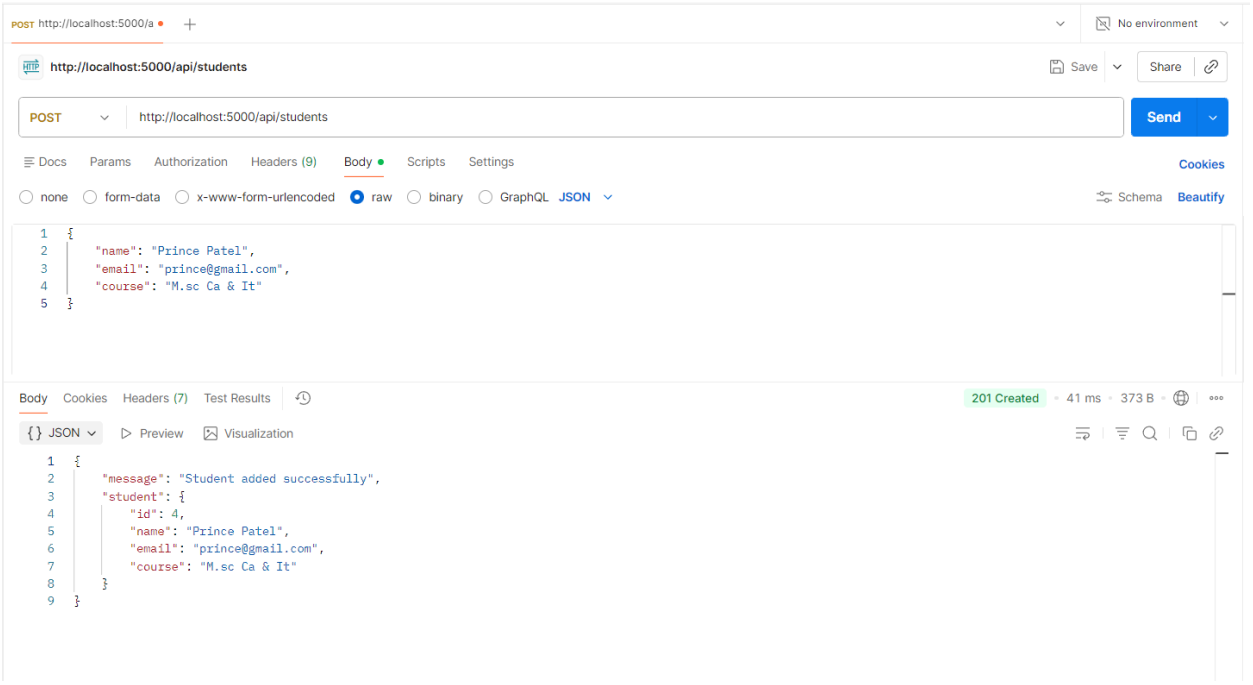
Pretty-print


```
[
  {
    "id": 1,
    "name": "Aryan Joshi",
    "email": "aryan@gmail.com",
    "course": "M.Sc. CA & IT"
  },
  {
    "id": 2,
    "name": "Divya Mehta",
    "email": "divya@gmail.com",
    "course": "M.Sc. CA & IT"
  },
  {
    "id": 3,
    "name": "Karan Sharma",
    "email": "karan@gmail.com",
    "course": "MCA"
  }
]
```

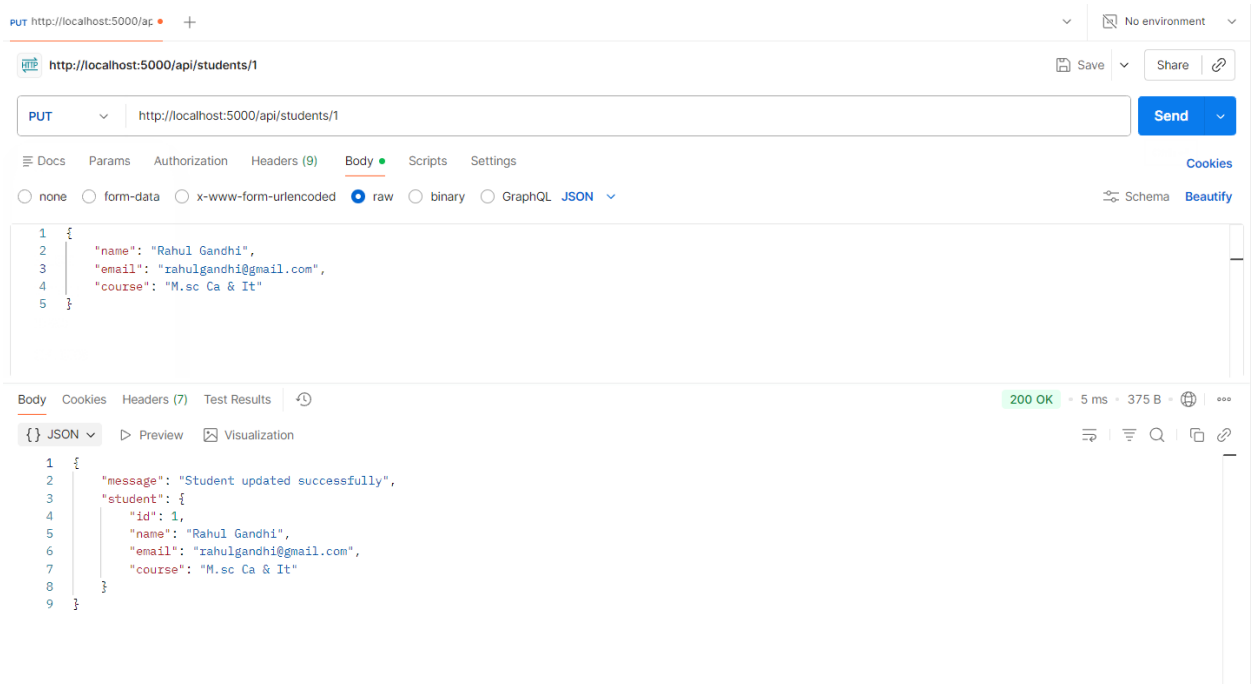

3) GET Student by ID



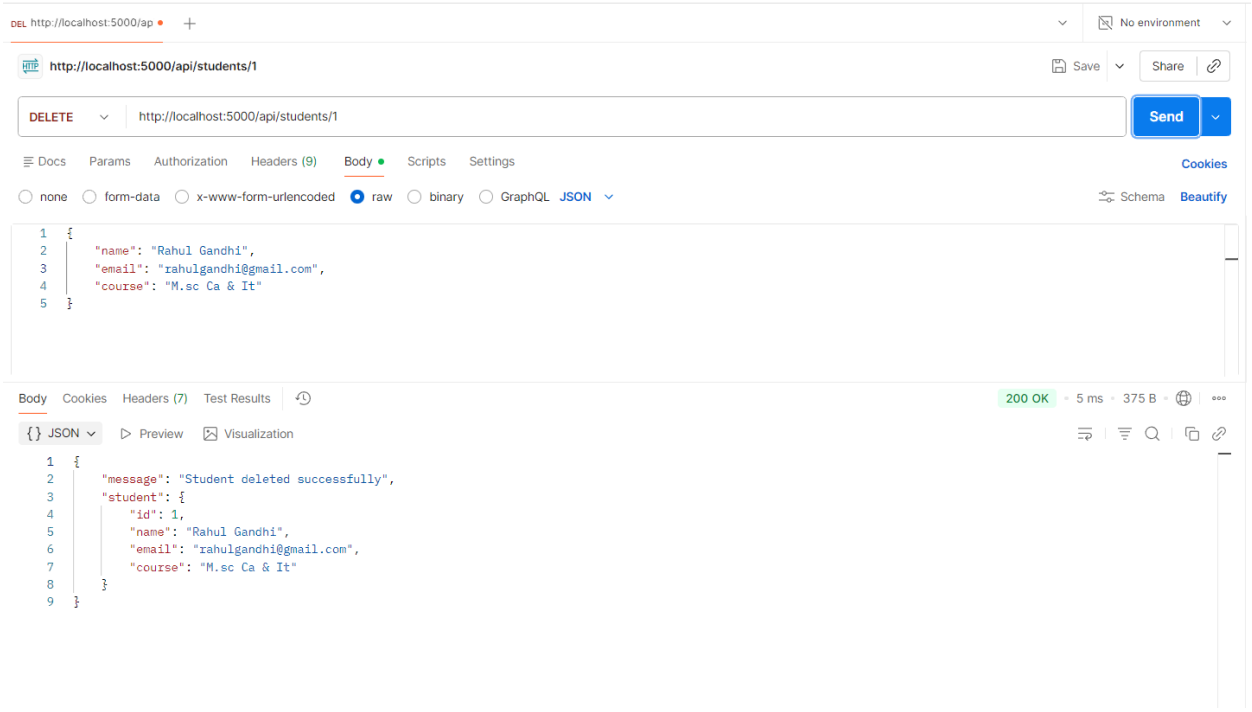
4) POST Student



5) PUT Student



6) DELETE Student



Result

Thus, RESTful APIs for Student Records Management were successfully developed using Express.js and tested using Postman. The application successfully performed CRUD operations such as retrieving, adding, updating and deleting student records.